

Protocolo TicAmazon

Formato de mensaje:

Cada mensaje sigue una estructura simple de comando

[ORIGEN -> DESTINO] TIPO MENSAJE <campo1> <campo2>...

Tamaño máximo del mensaje: 256 bytes

Campo	Tipo	Tam_Min	Tam_Max
tipo_mensaje	string	2	15 chars
categoria	string	3	20 chars
producto	string	1	50 chars
precio	float	-	8 chars (formato NNNNN.NN)
stock	int	1	5 digitos (0-99999)
descripcion	string	0	100 chars
count	int	1	3 digitos (0-999)

Expresiones regulares para validación

Campo	Regex
tipo_mensaje	^[A-Z_]{2,15}\$
categoría	^[a-zA-Z]{3,20}\$
producto	^[a-zA-Z0-9\-_]{1,50}\$
precio	^[0-9]{1,6}(\.[0-9]{1,2})?\$\$
stock	^[0-9]{1,5}\$
count	^[0-9]{1,3}\$

Cliente/Intermediario/Servidor valida los campos que recibe antes de procesarlos. Si algún campo no cumple con regex o tamaños determinados, salta mensaje de error.

a. Interacción Cliente – Intermediario (HTTP)

El cliente se comunica con el servidor intermedio mediante el protocolo HTTP, en el puerto TCP 80 (o 443 si se usa SSL). El cuerpo de las peticiones y respuestas HTTP transporta los mensajes definidos en este protocolo.

RE_CAT <string categoria> Cliente → Intermediario Solicita la lista de categorías disponibles

CAT_LIST:<lista> Intermediario → Cliente Responde con las categorías disponibles

RE_PROD:<categoria>	Cliente → Intermediario	Solicita los productos de una categoría
PROD:<lista>	Intermediario → Cliente	Responde con los productos disponibles
ADD_PROD:<id_producto>:<cantidad>	Cliente → Intermediario	Agrega un producto al carrito
FACTURA	Cliente → Intermediario	Solicita el cierre del pedido
FACTURA:<detalle>:<total>	Intermediario → Cliente	Responde con la factura proforma

b. Interacción Intermediario – Servidor de Bodega

Entre el servidor intermedio y los servidores de bodega se utiliza el mismo formato de mensajes, pero mediante un protocolo propio directo (sin HTTP), ya que esta comunicación no está sujeta al requisito de usar HTTP.

RE_CAT	Intermediario → Bodega	Solicita las categorías disponibles en esa bodega
CAT_LIST:<lista>	Bodega → Intermediario	Responde con las categorías
RE_PROD:<categoria>	Intermediario → Bodega	Solicita los productos de una categoría
PROD_LIST:<lista> disponibles	Bodega → Intermediario	Responde con los productos disponibles
RESERVE:<id_producto>:<cantidad>	Intermediario → Bodega	Reserva existencias para una orden en curso

1. Solicitud del listado de una categoría

Cliente → Intermediario
RE_CAT <string categoria>
Intermediario → Servidor
LIST_PROD <string categoria>
Servidor → Intermediario
PROD_LIST <int count> <string nombre,precio,stock; nombre,precio,stock; ...>
Intermediario → Cliente
PROD_LIST <int count> <string nombre,precio,stock; ...>

2. Si la categoría no tiene productos:

Servidor → Intermediario
PROD_LIST <int o>
Intermediario → Cliente
PROD_LIST_EMPT <string No hay productos disponibles en la categoría "cafeteria">

3. Detalle producto

Cliente → Intermediario
RE_PROD <string nombre_producto>
Intermediario → Servidor
FIND_PROD <string nombre_producto>
Servidor → Intermediario
PROD_FOUND <string nombre> <float precio> <int stock> <string descripcion>
Intermediario → Cliente
PROD <string nombre> <float precio> <int stock> <string descripcion>

4. Si no existe el producto

Servidor → Intermediario
PROD_NOT_FOUND <string nombre_producto>
Intermediario → Cliente
PROD_NOT_AVAIL <string nombre_producto no encontrado>

5. Agregar producto al carrito:

Cliente → Intermediario
ADD_CART <string producto> <int count>
Intermediario → Servidor
FIND_PROD <string producto>
Servidor → Intermediario
PROD_FOUND <string nombre> <float precio> <int stock> <string descripcion>
Intermediario → Cliente
PROD_ADDED <string producto> <float precio> <int count>

6. Si no hay stock suficiente para la cantidad pedida

Intermediario → Cliente
ERR_STOCK <string producto> <int stock_disponible> <int count_solicitado>

7. Solicitud de factura proforma

Cliente → Intermediario
GET_FACT <int count> <string producto,cantidad; producto,cantidad; ...>
Intermediario → Cliente
FACTURA <float total> <int count> <string producto,cantidad,subtotal; ...>

El intermediario arma la factura con los productos ya confirmados en anteriores pasos de ADD_CART (ya tiene precio y stock validados), no vuelve a consultar al servidor.

8. Si el carrito está vacío

Cliente → Intermediario
GET_FACT <int o>
Intermediario → Cliente

FACT_VACIA <string El carrito esta vacio>

9. Error de Formato – No cumple Regex

Detector → Sender

ERR_FORMAT <string tipo_mensaje_original> <string campo_invalido> <string valor_recibido>
--

10. Error de Tamaño

Detector → Sender

ERR_SIZE <string tipo_mensaje_original> <string campo> <int tamano_recibido> <int tamano_maximo>

11. Error de Formato – No cumple Regex

Detector → Sender

ERR_FORMAT <string tipo_mensaje_original> <string campo_invalido> <string valor_recibido>
--

12. Error de Tamaño

Detector → Sender

ERR_SIZE <string tipo_mensaje_original> <string campo> <int tamano_recibido> <int tamano_maximo>

13. Fallo Comunicación

Detector → Sender

ERR_COMM <string componente_no_responde> <string tipo_mensaje_pendiente> <int intentos>
--

Ejemplo salida:

[Cliente -> Intermediario] RE_CAT cafeteria

[Intermediario -> Servidor] LIST_PROD cafeteria

[Servidor -> Intermediario] PROD_LIST 4 items

[Intermediario -> Cliente] PROD_LIST 4 items
[Cliente -> Intermediario] RE_PROD "torta#3"
[Intermediario -> Cliente] ERR_FORMAT RE_PROD nombre_producto "torta#3"
[Intermediario -> Servidor] FIND_PROD tiramisu
[Servidor -> Intermediario] (sin respuesta después de 2 reintentos)
[Intermediario -> Cliente] ERR_COMM Servidor FIND_PROD 2
[Cliente -> Intermediario] ADD_CART CafeFrio 2
[Intermediario -> Servidor] FIND_PROD CafeFrio
[Servidor -> Intermediario] PROD_FOUND CafeFrio 3.25 40 "Cafe frio con hielo"
[Intermediario -> Cliente] PROD_ADDED CafeFrio 3.25 2
[Cliente -> Intermediario] GET_FACT 1 CafeFrio,2
[Intermediario -> Cliente] FACTURA 6.50 1 CafeFrio,2,6.50